

Problem Set 4

Peter Shi, Wenhui Xu

June 21, 2026

Question 1

Original problem statement.

Use one-sided finite differences to compute an approximation to the first derivative of

$$g(p) = 0.5p^{-0.5} + 0.5p^{-0.2}$$

at $p = 1.5$.

- (a) Let the increment ϵ in the finite differences range across all the values in the set $\{10^{-1}, 10^{-2}, \dots, 10^{-10}\}$. For which value of ϵ is the numerical first derivative most accurate?
- (b) Repeat part (a) using two-sided finite differences. How does the new value of ϵ compare?

We define

$$g(p) = 0.5p^{-0.5} + 0.5p^{-0.2}, \quad g'(p) = -0.25p^{-1.5} - 0.1p^{-1.2}.$$

Evaluating the analytical derivative at $p_0 = 1.5$ gives

$$g'(1.5) = -0.197556624253403.$$

Code

```
# --- Function definition ---
-
g(p)          = 0.5*p^(-0.5) + 0.5*p^(-0.2)
g_prime(p)    = -0.25*p^(-1.5) - 0.1*p^(-1.2)

p0            = 1.5
exact_deriv   = g_prime(p0)
epsilons      = [10.0^(-k) for k in 1:10]

@printf("Exact derivative at p = %.1f : %.15f\n", p0, exact_deriv)

# --- One-sided finite differences ---
-
one_sided(p, ε) = (g(p + ε) - g(p)) / ε

data_la = Matrix{Float64}(undef, length(epsilons), 3)

for (i, ε) in enumerate(epsilons)
    approx      = one_sided(p0, ε)
    abs_err     = abs(approx - exact_deriv)
```

```

        data_la[i, :] = [ε, approx, abs_err]
    end

    # — Two-sided finite differences —————
    two_sided(p, ε) = (g(p + ε) - g(p - ε)) / (2ε)

    data_lb = Matrix{Float64}(undef, length(epsilons), 3)

    for (i, ε) in enumerate(epsilons)
        approx      = two_sided(p0, ε)
        abs_err     = abs(approx - exact_deriv)
        data_lb[i, :] = [ε, approx, abs_err]
    end
end

```

Results for part (a): one-sided finite differences

Table 1: One-sided finite difference approximation to $g'(1.5)$

ϵ	Approx. $g'(1.5)$	Absolute Error
10^{-1}	-0.188764879271595	8.791745×10^{-3}
10^{-2}	-0.196635268930412	9.213553×10^{-4}
10^{-3}	-0.197464043121265	9.258113×10^{-5}
10^{-4}	-0.197547361658890	9.262595×10^{-6}
10^{-5}	-0.197555697944995	9.263084×10^{-7}
10^{-6}	-0.197556531578158	9.267524×10^{-8}
10^{-7}	-0.197556614400796	9.852607×10^{-9}
10^{-8}	-0.197556615511019	8.742384×10^{-9}
10^{-9}	-0.197556637715479	1.346208×10^{-8}
10^{-10}	-0.197556415670874	2.085825×10^{-7}

For one-sided finite differences, the approximation error is minimized at

$$\epsilon = 10^{-8},$$

with absolute error

$$8.742384 \times 10^{-9}.$$

As ϵ decreases from 10^{-1} to 10^{-8} , the approximation improves steadily. However, making ϵ even smaller (10^{-9} and 10^{-10}) worsens the approximation. This illustrates the standard trade-off between truncation error and numerical evaluation/rounding error: a smaller step size reduces truncation error at first, but once ϵ becomes too small, subtraction of very close numbers reduces numerical accuracy.

Results for part (b): two-sided finite differences

Table 2: Two-sided finite difference approximation to $g'(1.5)$

ϵ	Approx. $g'(1.5)$	Absolute Error
10^{-1}	-0.198056535804630	4.999116×10^{-4}
10^{-2}	-0.197561606653879	4.982400×10^{-6}
10^{-3}	-0.197556674075783	4.982238×10^{-8}
10^{-4}	-0.197556624751960	4.985571×10^{-10}
10^{-5}	-0.197556624259576	6.173201×10^{-12}
10^{-6}	-0.197556624281781	2.837769×10^{-11}
10^{-7}	-0.197556624392803	1.394000×10^{-10}
10^{-8}	-0.197556621062134	3.191269×10^{-9}
10^{-9}	-0.197556637715479	1.346208×10^{-8}
10^{-10}	-0.197556970782387	3.465290×10^{-7}

For two-sided finite differences, the approximation error is minimized at

$$\epsilon = 10^{-5},$$

with absolute error

$$6.173201 \times 10^{-12}.$$

Compared with the one-sided case, the value of ϵ that gives the best approximation is larger:

$$10^{-5} \quad \text{versus} \quad 10^{-8}.$$

At the same time, the minimum error is much smaller under the two-sided formula. This is consistent with the theory discussed in class and in the textbook: two-sided finite differences have truncation error of order h^2 , whereas one-sided finite differences have truncation error of order h . Therefore, the two-sided method is more accurate for moderate step sizes, although it requires more function evaluations.

Conclusion

The numerical results show that the increment ϵ should not be chosen to be arbitrarily small. For both one-sided and two-sided finite differences, the error first falls and then rises as ϵ becomes smaller. In this example, the most accurate one-sided approximation uses $\epsilon = 10^{-8}$, while the most accurate two-sided approximation uses $\epsilon = 10^{-5}$. Overall, the two-sided finite difference performs better, producing a substantially smaller error.

Question 2

Original problem statement.

Numerically find p_0 such that

$$g(p_0) = 0.75,$$

using:

- (a) Bisection,

- (b) Newton's method,
- (c) Brent's algorithm.

For each method, report the approximate value of p_0 , the computational effort required, and the runtime. For Newton's method and Brent's algorithm, use professionally written functions rather than writing the algorithms from scratch.

We rewrite the problem as a root-finding problem:

$$f(p) = g(p) - 0.75,$$

where

$$g(p) = 0.5p^{-0.5} + 0.5p^{-0.2}.$$

Thus, we seek p_0 such that

$$f(p_0) = 0.$$

For Newton's method, we also use the analytical derivative

$$f'(p) = g'(p) = -0.25p^{-1.5} - 0.1p^{-1.2}.$$

Code

```
# — Define residual —
-
f(p)      = g(p) - 0.75
f_prime(p) = g_prime(p) # derivative needed for Newton's method

# Quick bracket search
a, b = 1.0, 5.0
@printf("Bracket: f(%.1f) = %+.6f, f(%.1f) = %+.6f\n", a, f(a), b, f(b))
@assert f(a) * f(b) < 0 "No sign change      adjust bracket!"

# — Bisection (hand-coded; simple and transparent) —
function bisection(f, a, b; tol=1e-6, max_iter=1000)
    fa, fb = f(a), f(b)
    @assert fa * fb < 0 "f(a) and f(b) must have opposite signs."
    iters = 0
    local mid
    for i in 1:max_iter
        iters += 1
        mid = (a + b) / 2.0
        fmid = f(mid)
        if abs(fmid) < tol
            break
        elseif fa * fmid < 0
            b, fb = mid, fmid
        else
            a, fa = mid, fmid
        end
    end
    return mid, iters
end

p_bis, iters_bis = bisection(f, a, b; tol=1e-6)

# — Newton's method (Roots.jl) —
iter_count_newton = Ref{0}
f_tracked_newton(p) = (iter_count_newton[] += 1; f(p))
```

```

p_newt = find_zero((f_tracked_newton, f_prime), (a + b) / 2.0,
                  Roots.Newton(); atol=1e-6, rtol=0.0)

# — Brent's method (Roots.jl) —————
iter_count_brent = Ref{Int}()
f_tracked_brent(p) = (iter_count_brent[] += 1; f(p))

p_brent = find_zero(f_tracked_brent, (a, b), Roots.Brent();
                  atol=1e-6, rtol=0.0)

# — Timing comparison (100 repetitions each) —————
N_rep = 100

t_bis = @elapsed for _ in 1:N_rep
    bisection(f, a, b; tol=1e-6)
end

t_newt = @elapsed for _ in 1:N_rep
    find_zero(f, f_prime), (a + b) / 2.0, Roots.Newton(); atol=1e-6, rtol=0.0)
end

t_brent = @elapsed for _ in 1:N_rep
    find_zero(f, (a, b), Roots.Brent(); atol=1e-6, rtol=0.0)
end

```

Results

The initial bracket check gives opposite signs at the endpoints, so the interval $[1, 5]$ contains a root:

$$f(1) > 0, \quad f(5) < 0.$$

This makes bisection and Brent's method immediately applicable.

Table 3: Root-finding results for $f(p) = g(p) - 0.75$

Method	Approx. root p^*	Residual $f(p^*)$	Computational effort	Avg. time (μs)
Bisection	2.3275756836	4.75×10^{-7}	16 iterations	2.1065
Newton (<code>Roots.Newton()</code>)	2.3275726111	8.03×10^{-7}	4 function evaluations	875.3348
Brent (<code>Roots.Brent()</code>)	2.3275801726	-3.53×10^{-9}	8 function evaluations	1071.6745

Interpretation

All three methods converge to essentially the same root:

$$p^* \approx 2.32757.$$

Moreover, all three satisfy the required accuracy level, since the residuals are below 10^{-6} in absolute value. For bisection, the computational effort is reported as the number of iterations because the algorithm was coded directly. For Newton's method and Brent's algorithm, we used the professionally written implementations `Roots.Newton()` and `Roots.Brent()` from the Julia package `Roots.jl`. Since these library routines do not expose the same transparent iteration counter as our hand-coded bisection routine, we instead report the number of tracked function evaluations as a practical measure of computational effort.

Among the two library methods, Newton's method requires fewer function evaluations than Brent's algorithm in this example (4 versus 8), which is consistent with the usual fast local convergence of Newton's method

when the derivative is available and the initial guess is good. Brent's method requires more evaluations, but it delivers the smallest residual among the three methods and is generally regarded as a very robust derivative-free method.

In terms of raw wall-clock time, the hand-coded bisection routine appears much faster than the two library-based methods. However, this should not be interpreted as a general ranking of the algorithms. Here the problem is extremely small and cheap to evaluate, so package overhead is relatively important. In larger economic applications, the more relevant comparison is usually robustness and the number of expensive function evaluations rather than microsecond-level timing on a toy scalar problem.

Conclusion

The root of

$$g(p) = 0.75$$

is approximately

$$p^* \approx 2.32757.$$

Bisection is simple and reliable, but it requires more iterations. Newton's method is the most economical in terms of tracked function evaluations in this example, while Brent's algorithm is the most accurate in terms of the final residual and remains highly robust because it uses a bracket. Overall, the results are consistent with the standard trade-off discussed in class: bisection is safe but slow, Newton is fast but relies on derivative information, and Brent combines robustness with strong practical performance.

Question 3(a): Aiyagari (1994) Baseline with CRRA Preferences

Original problem statement.

This question asks us to solve the baseline model in Aiyagari (1994, QJE). We build on the consumption-savings (income fluctuation) decision problem solved in previous assignments and embed it in general equilibrium. Firms operate a Cobb-Douglas production function using capital and labor as inputs, while households supply capital through their optimal savings decisions. The capital market clears at the equilibrium interest rate.

The idiosyncratic income process follows an AR(1) process with persistence $\rho = 0.95$ and innovation standard deviation $\sigma_\eta = 0.2$. We discretize this process using the Rouwenhorst method with at least 9 states. We set the borrowing limit to zero and $\beta = 0.96$. This week, the problem can be solved by simulating decision rules and computing aggregates by averaging.

We first assume CRRA preferences and set risk aversion to 2. We find the stationary equilibrium capital stock and market-clearing interest rate. We then repeat the exercise for risk aversion 5, and compare the implied equilibrium capital stock and interest rate.

1. Model and computational strategy

We solve the household problem

$$V(k, z) = \max_{k'} \{u(c) + \beta E[V(k', z') \mid z]\},$$

subject to

$$c + k' = (1 + r)k + wz, \quad k' \geq 0,$$

where the CRRA utility function is

$$u(c) = \frac{c^{1-\gamma}}{1-\gamma}, \quad \gamma > 0, \gamma \neq 1.$$

The idiosyncratic income process is discretized using a 9-state Rouwenhorst approximation. Following the standard implementation, we interpret the discretized AR(1) states as log-income states and exponentiate them to obtain positive income levels. We then normalize mean income to one using the invariant distribution of the Markov chain.

On the production side, we use the Cobb-Douglas specification

$$Y = AK^\alpha L^{1-\alpha},$$

with baseline calibration

$$A = 1, \quad \alpha = 0.36, \quad \delta = 0.08, \quad L = 1.$$

Hence, for a given aggregate capital stock K , firm prices satisfy

$$r = A\alpha \left(\frac{K}{L}\right)^{\alpha-1} - \delta, \quad w = A(1-\alpha) \left(\frac{K}{L}\right)^\alpha.$$

Equivalently, given a candidate net interest rate r , firm capital demand is

$$K^d(r) = L \left(\frac{A\alpha}{r + \delta} \right)^{\frac{1}{1-\alpha}}.$$

Our computational strategy is:

1. For a candidate interest rate r , compute the implied firm wage $w(r)$ and capital demand $K^d(r)$.
2. Solve the household problem using brute-force VFI on a curved capital grid.
3. Simulate the induced savings policy and compute aggregate capital supply $K^s(r)$ by averaging across simulated households and time.
4. Define the excess capital function

$$F(r) = K^s(r) - K^d(r).$$

5. Use bisection on $F(r)$ to search for a market-clearing interest rate.

2. Key implementation blocks

2.1 Rouwenhorst discretization and curved capital grid

```

u_crra(c::Float64, γ::Float64) = c <= 0.0 ? -Inf :
    (γ == 1.0 ? log(c) : c^(1-γ)/(1-γ))

function expanding_grid(a, b, N, theta)
    t = range(0.0, 1.0; length=N)
    return a .+ (b-a) .* (t.^ theta)
end

function rouwenhorst(N::Int, ρ::Float64, σ_η::Float64)
    p = (1 + ρ) / 2
    q = p
    P = [1.0]

    for n in 2:N
        Pnew = zeros(n, n)
        Pnew[1:end-1, 1:end-1] .+= p * P
        Pnew[1:end-1, 2:end] .+= (1 - p) * P
        Pnew[2:end, 1:end-1] .+= (1 - q) * P
        Pnew[2:end, 2:end] .+= q * P
        if n > 2
            Pnew[2:end-1, :] ./= 2
        end
        P = Pnew
    end

    ψ = sqrt(N - 1) * σ_η / sqrt(1 - ρ^2)
    zgrid = collect(range(-ψ, ψ, length=N))
    return zgrid, P
end

```

2.2 Household problem given prices (r, w)

```

function household_solver_CRRA_VFI(; r::Float64, w::Float64,
    β::Float64 = 0.96, γ::Float64 = 2.0,
    Nz::Int = 9, ρ::Float64 = 0.95, σ_η::Float64 = 0.20,
    Nk::Int = 400, kmin::Float64 = 0.0, kmax::Float64 = 500.0,
    theta::Float64 = 1.0, tol::Float64 = 1e-6,
    maxit::Int = 10000, verbose::Bool = true)

```



```

inc = income_block(; Nz=Nz,  $\rho=\rho$ ,  $\sigma_\eta=\sigma_\eta$ )
zgrid_raw = inc.zgrid_raw
ygrid      = inc.ygrid
P           = inc.P
 $\pi z$        = inc. $\pi z$ 

kgrid = expanding_grid(kmin, kmax, Nk, theta)

V       = zeros(Nk, Nz)
V_new   = similar(V)
pol_idx = ones{Int, Nk, Nz}
policy_k = zeros(Nk, Nz)
policy_c = zeros(Nk, Nz)

it = 0
converged = false

while it < maxit
    it += 1
    EV = V * transpose(P)

    for iz in 1:Nz
        y = ygrid[iz]
        for ik in 1:Nk
            k = kgrid[ik]
            resources = (1 + r) * k + w * y

            best_val = -Inf
            best_j = 1

            for jk in 1:Nk
                kp = kgrid[jk]
                c = resources - kp
                val = u_crta(c,  $\gamma$ ) +  $\beta$  * EV[jk, iz]

                if val > best_val
                    best_val = val
                    best_j = jk
                end
            end

            V_new[ik, iz] = best_val
            pol_idx[ik, iz] = best_j
        end
    end

    dist = maximum(abs.(V_new .- V))
    V .= V_new

    if dist < tol
        converged = true
        break
    end
end

for iz in 1:Nz
    y = ygrid[iz]
    for ik in 1:Nk
        k = kgrid[ik]
        kp = kgrid[pol_idx[ik, iz]]
        policy_k[ik, iz] = kp
        policy_c[ik, iz] = (1 + r) * k + w * y - kp
    end
end

return (; V, policy_k, policy_c, pol_idx, kgrid, ygrid, P,  $\pi z$ , converged)
end

```

2.3 Simulation, firm block, and market clearing

```

function simulate_capital_supply(; hh, T::Int = 500, Nsim::Int = 1000,
                                burnin::Int = 200, seed::Int = 1234)

    Random.seed!(seed)

    kgrid = hh.kgrid
    pol_idx = hh.pol_idx
    P = hh.P
     $\pi$ z = hh. $\pi$ z

    Nk = length(kgrid)
    Nz = length(hh.ygrid)

    cdfP = cumsum(P, dims=2)
    cdf $\pi$  = cumsum( $\pi$ z)

    k_idx = ones(Int, Nsim)
    z_idx = similar(k_idx)

    for n in 1:Nsim
        u = rand()
        z_idx[n] = searchsortedfirst(cdf $\pi$ , u)
    end

    K_sum = 0.0
    count = 0

    for t in 1:(burnin + T)

        if t > burnin
            for n in 1:Nsim
                K_sum += kgrid[k_idx[n]]
                count += 1
            end

            k_idx_next = similar(k_idx)
            for n in 1:Nsim
                k_idx_next[n] = pol_idx[k_idx[n], z_idx[n]]
            end

            z_idx_next = similar(z_idx)
            for n in 1:Nsim
                u = rand()
                z_idx_next[n] = searchsortedfirst(view(cdfP, z_idx[n], :), u)
            end

            k_idx = k_idx_next
            z_idx = z_idx_next
        end

        return K_sum / count
    end

    const A_prod = 1.0
    const  $\alpha$ _prod = 0.36
    const  $\delta$ _prod = 0.08
    const L_prod = 1.0

    function firm_prices_from_K(K::Float64)
        r = A_prod *  $\alpha$ _prod * (K / L_prod)^( $\alpha$ _prod - 1) -  $\delta$ _prod
        w = A_prod * (1 -  $\alpha$ _prod) * (K / L_prod)^ $\alpha$ _prod
        return (; r, w)
    end
end

```

```

function firm_K_demand_from_r(; r::Float64)
    return L_prod * (A_prod *  $\alpha_{\text{prod}}$  / (r +  $\delta_{\text{prod}}$ ))^(1 / (1 -  $\alpha_{\text{prod}}$ ))
end

function capital_supply_from_r(; r::Float64,  $\gamma$ ::Float64,
     $\beta$ ::Float64 = 0.96, Nz::Int = 9,  $\rho$ ::Float64 = 0.95,  $\sigma_{\eta}$ ::Float64 = 0.20,
    Nk::Int = 400, kmin::Float64 = 0.0, kmax::Float64 = 500.0,
    theta::Float64 = 1.0, hh_tol::Float64 = 1e-6, hh_maxit::Int = 10000,
    T::Int = 500, Nsim::Int = 1000, burnin::Int = 200, seed::Int = 1234)

    Kd = firm_K_demand_from_r(r = r)
    prices = firm_prices_from_K(K = Kd)
    w = prices.w

    hh = household_solver_CRRA_VFI(
        r = r, w = w,  $\beta$  =  $\beta$ ,  $\gamma$  =  $\gamma$ , Nz = Nz,  $\rho$  =  $\rho$ ,  $\sigma_{\eta}$  =  $\sigma_{\eta}$ ,
        Nk = Nk, kmin = kmin, kmax = kmax, theta = theta,
        tol = hh_tol, maxit = hh_maxit, verbose = false
    )

    Ks = simulate_capital_supply(hh = hh, T = T, Nsim = Nsim,
        burnin = burnin, seed = seed)

    return (; Ks, Kd, w, hh)
end

function excess_capital_from_r(; r::Float64,  $\gamma$ ::Float64, kwargs...)
    out = capital_supply_from_r(r = r,  $\gamma$  =  $\gamma$ ; kwargs...)
    excess = out.Ks - out.Kd
    return (; excess, Ks = out.Ks, Kd = out.Kd, w = out.w, hh = out.hh)
end

```

2.4 Bisection on the excess capital function

We reused the same bisection routine from Question 2. The only change is that the scalar function is now the excess capital function $F(r) = K^s(r) - K^d(r)$.

```

function bisection(f, a, b; tol=1e-6, max_iter=1000)
    fa, fb = f(a), f(b)
    @assert fa * fb < 0 "f(a) and f(b) must have opposite signs."
    iters = 0
    local mid
    for i in 1:max_iter
        iters += 1
        mid = (a + b) / 2.0
        fmid = f(mid)
        if abs(fmid) < tol
            break
        elseif fa * fmid < 0
            b, fb = mid, fmid
        else
            a, fa = mid, fmid
        end
    end
    return mid, iters
end

```

3. Equilibrium for $\gamma = 2$

For risk aversion $\gamma = 2$, a positive-interest-rate bracket was available. Using the interval $[0.01, 0.04]$, we solved the market-clearing problem by bisection using

$$F(r) = K^s(r) - K^d(r).$$

```
r_low = 0.01
r_high = 0.04

r_bis, iters_bis = bisection(
  r -> excess_capital_from_r(r = r,  $\gamma$  = 2.0,
    T = 500, Nsim = 1000, burnin = 200,
    Nk = 400, kmax = 500.0, theta = 1.0).excess,
  r_low, r_high;
  tol = 1e-2, max_iter = 30
)

out_star = excess_capital_from_r(
  r = r_bis,  $\gamma$  = 2.0,
  T = 500, Nsim = 1000, burnin = 200,
  Nk = 400, kmax = 500.0, theta = 1.0
)
```

The resulting approximate equilibrium is:

$$r^* \approx 0.014728, \quad K^s(r^*) \approx 8.122669, \quad K^d(r^*) \approx 8.053362, \quad w(r^*) \approx 1.356225.$$

The remaining excess capital is

$$K^s(r^*) - K^d(r^*) \approx 0.069308,$$

which is small relative to the aggregate capital stock, so we treat this as a numerically close market-clearing solution.

4. Equilibrium for $\gamma = 5$

For risk aversion $\gamma = 5$, the positive-interest-rate interval did not bracket a root: the excess capital function remained positive throughout the positive range we initially considered. This suggested that the equilibrium net interest rate is lower than in the $\gamma = 2$ case. Since firm capital demand is well-defined as long as $r + \delta > 0$, we expanded the search to moderately negative net interest rates.

A coarse scan over the interval $[-0.06, 0.02]$ showed a sign change:

$$F(-0.0236) < 0, \quad F(-0.0091) > 0.$$

We then applied bisection on this bracket.

```
r_low_5 = -0.0236
r_high_5 = -0.0091

r_bis_5, iters_bis_5 = bisection(
  r -> excess_capital_from_r(r = r,  $\gamma$  = 5.0,
    T = 300, Nsim = 500, burnin = 100,
    Nk = 300, kmax = 500.0, theta = 1.0).excess,
  r_low_5, r_high_5;
```

```

    tol = 1e-4, max_iter = 30
)

out_star_5 = excess_capital_from_r(
    r = r_bis_5,  $\gamma$  = 5.0,
    T = 300, Nsim = 500, burnin = 100,
    Nk = 300, kmax = 500.0, theta = 1.0
)

```

The resulting approximate equilibrium is:

$$r^* \approx -0.015059, \quad K^s(r^*) \approx 14.675708, \quad K^d(r^*) \approx 14.526400, \quad w(r^*) \approx 1.677092.$$

The remaining excess capital is

$$K^s(r^*) - K^d(r^*) \approx 0.14931.$$

We emphasize that this $\gamma = 5$ result was obtained using a lighter “fast-pass” configuration ($T = 300$, $Nsim = 500$, $burnin = 100$, $Nk = 300$) in order to keep the outer root-finding computationally feasible. Therefore, the $\gamma = 5$ result should be interpreted as an approximate equilibrium obtained from the same algorithmic pipeline, but with a coarser numerical configuration than the $\gamma = 2$ case.

5. Comparison and interpretation

Table 4: Approximate stationary equilibria under CRRA preferences

Risk aversion γ	r^*	$K^s(r^*)$	$K^d(r^*)$	Excess	$w(r^*)$
2	0.014728	8.122669	8.053362	0.069308	1.356225
5	-0.015059	14.675708	14.526400	0.149310	1.677092

The comparison shows a clear difference between the two cases. When risk aversion rises from 2 to 5, the equilibrium capital stock increases substantially, while the market-clearing net interest rate falls sharply. In our computation, the net interest rate moves from a positive value in the $\gamma = 2$ case to a negative value in the $\gamma = 5$ case (while remaining above $-\delta$, so firm capital demand remains well-defined).

The economic intuition is precautionary saving. A higher value of γ makes households more averse to idiosyncratic income risk. Since markets are incomplete and households cannot fully insure against bad income realizations, they respond by accumulating more capital for self-insurance. This shifts the aggregate capital supply curve outward. In general equilibrium, the additional capital supply depresses the equilibrium interest rate and raises the stationary capital stock.

Thus, within this baseline Aiyagari environment, raising CRRA risk aversion from 2 to 5 generates a strong increase in savings and a lower market-clearing return on capital. The difference is economically large and clearly visible in the computed equilibrium objects.

6. Summary

Using a 9-state Rouwenhorst approximation for idiosyncratic income risk, brute-force VFI for the household problem, simulation-based aggregation for capital supply, and bisection on the excess capital function, we computed approximate stationary equilibria for the baseline Aiyagari model under two CRRA preference parameterizations.

For $\gamma = 2$, we obtain

$$r^* \approx 0.0147, \quad K^* \approx 8.1.$$

For $\gamma = 5$, we obtain

$$r^* \approx -0.0151, \quad K^* \approx 14.7.$$

Hence, higher risk aversion leads to more precautionary saving, higher aggregate capital, and a lower equilibrium interest rate.